SADINhub ML — how to run it

SADINhub ML - operator guide - what to run, from where, and what is gated

Everything below runs from **one folder**:

```
C:\Users\shafe\OneDrive\Desktop\SadinHub_ Project\ML_WORK
```

Open PowerShell there. Every command in this document assumes that is your working directory.

---

0. One-time setup

The virtual environment already exists at `.venv`. Use its interpreter directly — you do not need to activate anything:

```
.\.venv\Scripts\python.exe --version
```

If you ever need to rebuild it:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

The database password

It is never stored in the code, the config, the logs, or the run metadata. Any stage that touches PostgreSQL asks for it one of two ways:

Method	How
Environment variable	`$env:SADINHUB_PASSWORD = "..."` before running
Interactive prompt	just run the script — it will ask, and the typing is hidden

The prompt is the safer of the two, because the environment variable stays in your shell history and in the process table. Use the variable only when you are running several stages back to back.

Every connection is opened **read-only** and **REPEATABLE READ**, and the code verifies `transaction_read_only = on` before it runs a single query. If the server ever came back writable, the run aborts rather than continuing.

---

1. The fastest way to check everything still works

This needs **no database and no password**:

```
.\.venv\Scripts\python.exe -m pytest tests\ -q
```

Expected: **272 passed, 4 skipped** in about 30 seconds.

The 4 skips are the live-database replay tests. They skip because `SADINHUB_PASSWORD` is not set — that is correct behaviour, not a failure. To run those 4 as well, set the password variable first and re-run.

---

2. What is already built, and where it lives

Nothing below needs to be re-run. These are the frozen results:

Stage	Output folder	State
Task 0A / 0B — leakage and label audit	`outputs/task0a_...`, `outputs/task0b_...`	complete
Model 2 — daily kWh quantile forecaster	`outputs/model2_20260918T130515Z`	**FROZEN. Final test CONSUMED — do not re-run**
Model 1 — raw-reading anomaly detection	`outputs/model1_20260920T080937Z`	**FROZEN**
Model 3 — features	`outputs/model3_features_20260920T083853Z`	**FROZEN**
Model 3 — triage scoring	`outputs/model3_20260920T085615Z`	**FROZEN**
Task 4 — engineering review package	`outputs/task4_20260920T102354Z`	**ASSIGNMENT FROZEN — DRAFT**

`config/task4_config.yaml` points at those exact folder names under `frozen_runs:`. If you re-run a model, it writes a **new timestamped folder** and does not touch the old one — but Task 4 will keep using the frozen one until you edit that config block.

---

3. Running each stage

Scripts are run by path. You do not need `-m` or any `PYTHONPATH` setting.

Model 1 — anomaly detection on raw readings · needs the database

```
.\.venv\Scripts\python.exe src\model1_anomaly_raw\run_model1.py
```

Useful while testing, to process only the first few meters:

```
.\.venv\Scripts\python.exe src\model1_anomaly_raw\run_model1.py --limit-meters 3
```

Takes roughly 10–15 minutes for the full 31-meter scope. Writes a new `outputs/model1_<timestamp>/` with `findings_summary.csv`, `anomaly_events.parquet`, `results_summary.md` and `run_metadata.json`.

Model 2 — forecaster · needs the database · do not re-run

```
# For reference only. Do not execute without a decision to un-freeze Model 2.
.\.venv\Scripts\python.exe src\model2_forecaster\run_model2.py
```

Model 2 is historically evaluated and frozen. Its May 12 – July 10 2026 final test is marked `CONSUMED` in the run metadata and **must not be reused for model selection**. Re-running it and looking at that window again would quietly turn a held-out test into a validation set, and the 11.76% WAPE would stop meaning what it currently means.

Model 3 — triage · features need the database, the rest do not

Run these **in order**:

```
# 1. build the feature table                          (needs the database)
.\.venv\Scripts\python.exe src\model3_triage\run_features.py

# 2. fit Isolation Forest and score every meter-day    (offline)
.\.venv\Scripts\python.exe src\model3_triage\run_model3.py

# 3. cross-check the ranking against Model 1           (offline)
.\.venv\Scripts\python.exe src\model3_triage\run_crosscheck.py

# 4. meter-preserving permutation null                 (offline)
.\.venv\Scripts\python.exe src\model3_triage\run_null_correction.py
```

Steps 2–4 read the frozen artifacts from step 1, so you can re-run them freely without touching the database.

Task 4 — build the review queue · offline

```
.\.venv\Scripts\python.exe src\review_loop\build_queue.py
```

Reconciles the meter-day populations, freezes the allocation table **before** drawing any row, draws the 200 units from seed `20260926`, and writes the blinded workbooks plus the withheld key. Deterministic: same seed, same 200 units, every time.

Task 4 — issue the workbooks · offline

```
.\.venv\Scripts\python.exe src\review_loop\issue_workbooks.py --run outputs\task4_20260920T102354Z
```

This is the stage that is currently gated. It runs all nine assignment verifications, rebuilds the two workbooks, and then stops:

```
STATUS: placeholder identities still present - workbooks are written as a
        DRAFT and must not be issued until real names are configured.
```

It exits with **code 2** and writes `..._DRAFT.xlsx`. That is the designed behaviour, not an error. See section 5.

---

4. Reading the results

You want	Open
What the models are and what they found	`docs/01` – `docs/06` PDFs, in that order
Model 1 findings, one row per finding	`outputs/model1_20260920T080937Z/findings_summary.csv`
Model 3 ranked meter-days	`outputs/model3_20260920T085615Z/`
Which 200 units go to review	`outputs/task4_20260920T102354Z/task4_reviewer_assignment.csv`
The engineer-facing workbooks	`outputs/task4_20260920T102354Z/task4_labelling_sample_*_DRAFT.xlsx`
Un-blinded worklist, all 405 findings	`outputs/task4_20260920T102354Z/task4_worklist.xlsx`
Proof of what a run did	`run_metadata.json` in any run folder

Do not open the withheld key (`task4_queue_key.parquet`) in front of a reviewer. It maps each blinded `review_id` back to its meter, date, family and model flags. Showing it to someone who is about to label would destroy the blinding the whole estimate depends on.

---

5. The one thing still blocking issuance

The package cannot be sent to engineering until three real names exist. Edit `config/task4_config.yaml`:

```
governance:
  adjudicator: "TBD - must be named before the workbooks are issued"
  reviewers: ["reviewer_1", "reviewer_2"]
```

Replace all three placeholders with the real engineering identifiers, then re-run:

```
.\.venv\Scripts\python.exe src\review_loop\issue_workbooks.py --run outputs\task4_20260920T102354Z
```

It will then exit **0**, drop the `_DRAFT` suffix, and mark the package `issuable`.

Nothing is redrawn by this. The 200 units, the inclusion probabilities and the reviewer assignment are all reproduced from the frozen seed. Verified: on a repeat run, every sampling and assignment artifact comes back byte-identical, and the two workbooks differ only in their embedded creation timestamp — the 220 rows inside them are the same rows.

Why the gate exists: a placeholder reviewer id makes the 20 double-labelled units un-attributable, so no agreement can be computed from them; and an unnamed adjudicator leaves every `Unsure` and every disagreement with no owner to resolve it.

---

6. What must not be computed yet

No precision, no `unflagged_fault_rate`, no Model 3 validation, no agreement — because no label exists. All four wait for completed workbooks to come back from engineering.

When they do return, the estimation stage is separate and already written: `src/review_loop/estimation.py` and `src/review_loop/agreement.py`. The reported precision interval is **survey-weighted linearisation with a finite-population correction**; the FPC-aware stratified bootstrap is a secondary cross-check; a simple binomial interval is never reported.

---

7. Standing rules this codebase enforces

These are not style preferences — the code refuses to violate them:

- PostgreSQL access is **read-only**, verified at connect time, `REPEATABLE READ`.
- **Nothing** is written to Bronze, Silver, Gold or Power BI. No tables are created.
- All outputs stay inside `ML_WORK/`.
- The database password never reaches source, a DSN, a log, or run metadata — `write_run_metadata` refuses any payload containing the substring `password`.
- A data-quality or electrical business rule is never changed to make a model look better.

---

8. If something goes wrong

Symptom	Cause
`No module named pytest`	You used the system `python`. Use `.\.venv\Scripts\python.exe`.
`Refusing to proceed: transaction_read_only is 'off'`	The server is not read-only. **Do not override this.** Investigate the connection.
`issue_workbooks.py` exits 2	Correct — placeholder identities. See section 5.
4 tests skipped	Correct — `SADINHUB_PASSWORD` not set.
Task 4 uses an old model run	`frozen_runs:` in `config/task4_config.yaml` pins it deliberately.